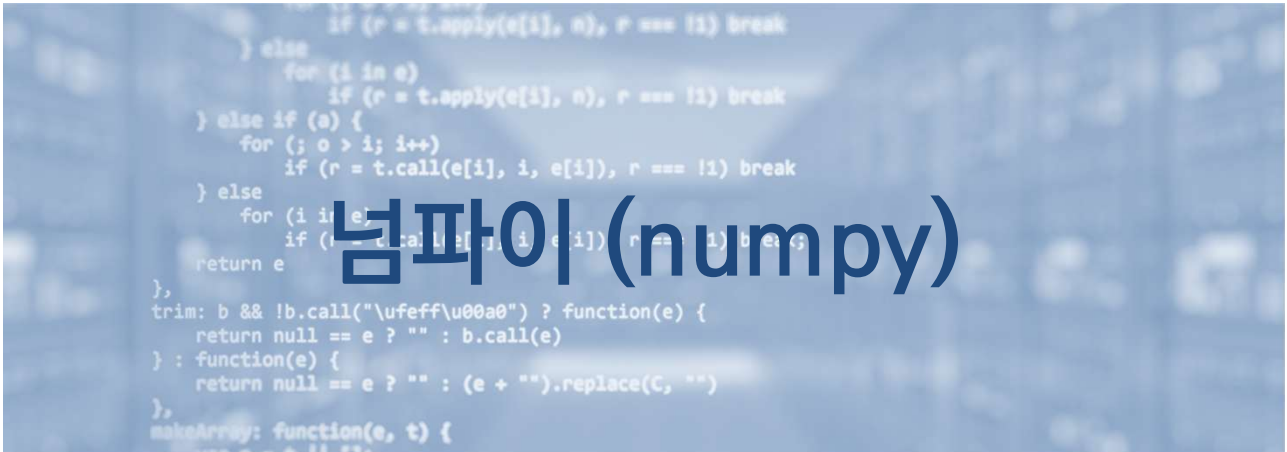




한국폴리텍대학 성남캠퍼스 인공지능소프트웨어과  
이혜정 교수

1

넘파이 (NumPy : Numerical Python)

- 수치 데이터를 다루는 파이썬 패키지
  - 배열 계산에 최적화된 방식
    - Loop를 사용하지 않고 대량 데이터의 배열 연산을 가능하게 함
  - 처리 속도가 빠름
  - 다차원 리스트나 크기가 큰 데이터 처리에 유리
  - 머신러닝, 딥러닝 알고리즘의 입력 데이터와 출력 데이터를 넘파이 배열 타입으로 사용
- [Numpy Reference](#)
- 사용
  - numpy 모듈 설치 `conda install numpy`
  - numpy 사용
    - np라는 별칭 사용 `import numpy as np`
  - 버전 확인 `np.__version__`

※ 본 자료는 2026년 2학기 『AI기반데이터분석(이혜정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

2

## 데이터 분석에 사용하는 데이터형

### ■ 리스트, list

- 파이썬 데이터 분석에서 가장 많이 사용하는 기본 데이터형
- 다양한 값을 연속으로 저장하는 것

### ■ 넘파이 배열, ndarray

- 넘파이 라이브러리에서 제공하는 ndarray 형태의 데이터형
- ndarray는 n-dimension array(n-차원 배열)이라는 의미
- 데이터 분석 처리에 주로 사용

### ■ 데이터프레임, dataframe

- 판다스 라이브러리에서 제공하는 데이터형
- 데이터 전처리에 주로 사용

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

3

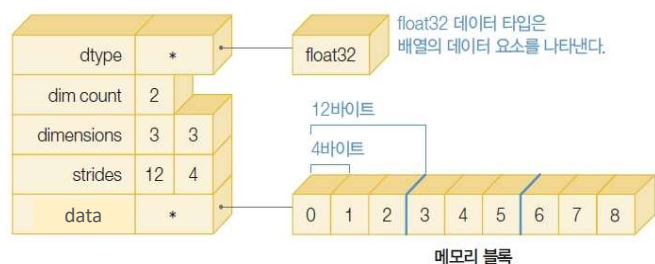
3

## 넘파이 배열, ndarray

### ■ 반복문 없이도 대규모 수치 계산을 빠르게 처리할 수 있도록 설계된 n-차원 배열 객체

| 리스트                                                                                                            | 넘파이 배열                                                                                                          |
|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• 동작 속도가 느림</li> <li>• 쉽표로 요소 구분</li> <li>• 서로 다른 데이터형 사용 가능</li> </ul> | <ul style="list-style-type: none"> <li>• 동작 속도가 빠름</li> <li>• 공백으로 요소 구분</li> <li>• 한 가지 데이터형만 사용 가능</li> </ul> |

- 동일한 자료형 (Homogeneous)
  - 배열 내의 모든 원소는 반드시 같은 데이터 타입
- 연속된 메모리 배치 → 접근 속도가 빠름
- 벡터화 연산 (Vectorization)
  - 반복문(for문) 없이 배열 전체에 대해 한 번에 계산을 수행



※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

4

4

## 넘파이 배열의 주요 속성

| 속성    | 설명               | 속성     | 설명            |
|-------|------------------|--------|---------------|
| dtype | 데이터형 표시          | T      | 배열의 행/열 방향 변경 |
| shape | 배열이 몇 행 몇 열인지 표시 | size   | 배열의 원소 수 표시   |
| ndim  | 배열의 차원 표시        | nbytes | 배열의 byte 표시   |
| flat  | 배열의 값 초기화        |        |               |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

5

5

## 넘파이 배열의 주요 데이터 타입

| 데이터 타입 | NumPy dtype               | 설명            | 예시                        |
|--------|---------------------------|---------------|---------------------------|
| 정수형    | int8, int16, int32, int64 | 소수점이 없는 정수 저장 | 10, -3, 100               |
| 실수형    | float16, float32, float64 | 소수점이 있는 실수 저장 | 3.14, -0.5                |
| 논리형    | bool                      | 참/거짓 값 저장     | True, False               |
| 문자열형   | str_, <U...               | 유니코드 문자열 저장   | 'Python', 'Kim'           |
| 객체형    | object                    | Python 객체를 저장 | 10, 'Kim', [1, 2] 등 혼합 가능 |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

6

6

## n차원 넘파이 배열

### ■ 넘파이 배열은 n차원으로 구성할 수 있음

- 1차원, 2차원, 3차원이 대표적임
  - 1차원 : 하나의 행 또는 열을 가진 배열 (Rank1으로 표현)
    - ▶ shape 함수를 사용하여 구조를 출력하면 (n,) 형태로 표시
  - 2차원 : 행과 열로 이루어진 배열 (Rank2로 표현)
  - 3차원 : 2차원 배열이 여러 개인 형태 (Rank 3로 표현)
    - ▶ 행과 열로 이루어진 면이 여러 개인 형태

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
|---|---|---|---|

(a) 1차원 배열: (x,)

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 |

(b) 2차원 배열: (x,y) / 행렬(matrix)

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 |

(c) 3차원 배열: (x,y,z) / (면행열)

- 첫 데이터부터 마지막 데이터까지 대괄호 개수를 통해 차원을 구분
- 배열의 차원의 구조는 shape 속성으로 확인

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

7

7

## 넘파이 배열의 축(Axis)

### ■ 축(Axis)을 추가하여 차원 확장

- 기존 배열에 새로운 방향을 추가

| 용어               | 설명           | 예시              |
|------------------|--------------|-----------------|
| Rank (Dimension) | 축의 개수        | 1D, 2D, 3D ...  |
| Axis 0           | 행(Row) 방향    | 위에서 아래로         |
| Axis 1           | 열(Column) 방향 | 왼쪽에서 오른쪽으로      |
| Axis 2           | 깊이(Depth) 방향 | 앞에서 뒤로 (3차원 이상) |

|          |          |    |    |    |
|----------|----------|----|----|----|
|          | axis=1 → |    |    |    |
| axis=0 ↓ | 1        | 2  | 3  | 4  |
|          | 5        | 6  | 7  | 8  |
|          | 9        | 10 | 11 | 12 |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

8

8

## 넘파이 배열의 생성

### ■ 기존 데이터로부터 넘파이 배열 생성

- `ndarray = np.array(data, dtype=None)`
  - `data` : array\_like (리스트, 튜플, 데이터프레임)
  - `dtype` : 데이터 타입

- `data`가 동일 타입이 아닌 경우, 동일 타입으로 변환

```
a1 = np.array([1, 2, 3])
a2 = np.array((1, 2, 3))
a3 = np.array([[1, 2, 3], [4, 5, 6]])
a4 = np.array([1, 2, 3], dtype=float)
a5 = np.array([1, 2, 3], float)
a6 = np.array([1, 4, 5, "8"])
a7 = np.array([1, 4, 5, "8"], int)
```

```
test_list = [1, 4, 5, 8, 'a']
np.array(test_list, int) # ValueError
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

9

9

## 넘파이 배열의 생성

### ■ `arange()` 함수를 이용하여 넘파이 배열 생성

- 특정 수열로 1차원의 넘파이 배열을 생성

- `ndarray = np.arange(시작값, 종료값, 증가값)`
- `ndarray = np.arange(start=시작값, stop=종료값, step=증가값)`
  - 시작값부터 증가값만큼씩 커지는 배열을 종료값 전까지 생성 (파이썬 `range` 함수와 동일)
  - 증가값 생략 가능 -> 1로 처리
  - 종료값만 작성 가능 -> 시작값은 0으로, 증가값은 1로 처리

```
a1 = np.arange(10)
a2 = np.arange(2,6)
a3 = np.arange(1,11,2)

a4 = np.arange(start=1, stop=11, step=1)
a5 = np.arange(stop=11, step=2)
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

10

10

## 넘파이 배열의 데이터형 확인과 변형

### ■ 데이터형 확인 : type 함수, dtype 속성

- type 함수 : 소괄호 안의 인자나 값의 데이터형 반환
- dtype 속성 : 배열 요소의 데이터형을 반환

```
type(데이터)  
배열명.dtype
```

### ■ 데이터형 변환 : astype 함수 사용

```
배열명.astype(변경할 데이터형)
```

```
nda=np.array([0, 1, 2, 3, 4, 5])  
type(nda)  
nda.dtype  
  
a1 = np.array(['1.2', '2.3', '3.4', '4.5'])  
a2 = a1.astype(float)  
a3 = a2.astype(int)  
  
a4 = a1.astype(int) -> ValueError
```

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

11

## 배열의 형태 변경

### ■ 배열의 구조 변경 : reshape 함수

- 1차원 배열을 2차원 배열로, 2차원 또는 1차원 배열을 3차원 배열로 변경 가능

```
배열명.reshape(행 수, 열 수)  
배열명.reshape(면 수, 행 수, 열 수)
```

- 배열의 면 수, 행 수, 열 수 중 하나를 자동으로 조정하기 원할 때는 -1로 설정
  - 행 수를 -1로 설정하고 열 수를 지정하면,  
열의 개수는 지정한 값으로 맞추고 행 수는 전체 원소 수에 맞춰 표시  
(3차원 배열도 동일 함)

### ■ 다차원 배열을 1차원 배열로 평탄화 : flatten 함수

```
a1 = np.arange(6)  
a2 = a1.reshape(6, 1)  
a3 = a1.reshape(2, 3)  
  
b1 = np.arange(12)  
b2 = b1.reshape(2, 2, 3)  
  
c1 = np.arange(6)  
c2 = c1.reshape(-1, 2)  
c3 = c1.reshape(2, -1)  
c4 = c1.reshape(-1, 2, 3)  
  
a = np.arange(12).reshape(3, 4)  
f = a.flatten()
```

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

12

## 배열의 형태 변경

### 배열의 방향 변경

|                          |                              |
|--------------------------|------------------------------|
| 배열명[::-1]                | 행을 기준으로 배열을 뒤집어 표시           |
| 배열명[:,::-1]              | 열을 기준으로 배열의 좌/우를 뒤집어 표시      |
| 배열명[::-1,::-1]           | 배열의 상하좌우가 모두 거꾸로 변경되어 표시     |
| 배열명.transpose()<br>배열명.T | 배열의 행과 열 방향을 변경하여 표시 (전치 행렬) |

```
a=np.arange(12).reshape(3, 4)
b1 = a[::-1]
b2 = a[:,::-1]
b3 = a[::-1,::-1]
b4 = a.transpose()
b5 = a.T
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

13

## 인덱싱 (indexing)

### 리스트에 있는 값의 상대적인 주소(offset)로 값에 접근하여 원하는 값을 표시하거나 변경하는 것

- 리스트의 indexing 방식과 동일
- 2차원 이상의 배열 인덱스 표현에는 ‘,’을 지원
  - 2차원 : [행][열] 또는 [행, 열]
  - 3차원 : [면][행][열] 또는 [면, 행, 열]
  - 3차원 이상은 shape에서 출력되는 랭크 순서대로 인덱싱에 접근

```
d=np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
print(d[0], d[1], d[2], d[-1], d[-2])
```

```
d2 = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(d2[0][0]) # 배열명[행][열] 형태
print(d2[0,0]) # 배열명[행,열] 형태
print(d2[1,1])
print(d2[1, -1])
```

```
d3 = np.arange(12).reshape(2, 2, 3)
print(d3[0][0][0]) # 배열명[면][행][열] 형태
print(d3[0, 0, 0]) # 배열명[면,행,열] 형태
print(d3[1, 1, 2])
print(d3[-1, -1, -1])
```

```
d[3] = 100
d2[1, 1] = 100
d3[1, 1, 1] = 1000
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

14

14

## 슬라이싱 (slicing)

### ■ 인덱스를 사용하여 리스트 일부를 잘라내어 반환

- 리스트 slicing 방식과 동일

배열명[start:stop:step, ... ]



그림 8-2 2차원 배열의 슬라이싱

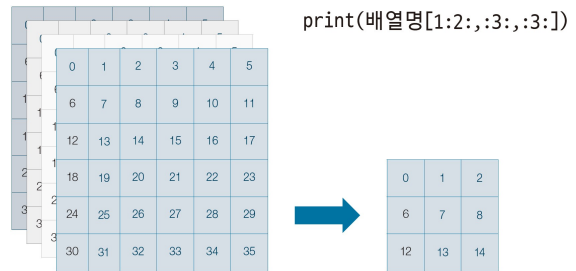


그림 8-3 3차원 배열의 슬라이싱

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

15

15

## 슬라이싱 (slicing)

```
d=np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
print(d)
print(d[2:3], d[:,], d[:-1])
print(d[::2], d[:::-1])
```

```
d2 = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(d2[0]) # 0행만 슬라이싱
print(d2[0,:]) # 0행의 전체 열 슬라이싱
print(d2[:, :-1]) # 마지막 열 제외한 전체 행과 열 슬라이싱
print(d2[::2, ::2]) # 2행 2열 간격으로 전체 행과 열 슬라이싱
```

```
# 슬라이싱된 요소값을 한꺼번에 변경
d2 = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(d2)
print(d2[::2, ::3])
d2[::2, ::3] = 30
print(d2)
```

```
# 전체 요소값을 한꺼번에 변경
d2[:, :] = -1
print(d2)
```

```
d3 = np.arange(12).reshape(2, 2, 3)
print(d3)
print(d3[:, :-1, :-1])
print(d3[:, 1, 1:])
print(d3[:, :, -1])
print(d3[:, -1, -1])
```

```
d2[0] = 50
d3[0] = 50
d3[1,0] = 30
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

16

16

## 불 인덱싱 (Boolean indexing)

### ■ 조건을 이용한 인덱싱

- 조건 검색 및 필터를 사용하여 데이터를 추출할 수 있음

배열명[조건식]

- 조건에 맞는 결과만 배열에 포함
  - 조건으로 설정된 값을 인덱스로 사용하여 조건에 맞는 값만 결과로 표시할 수 있음
  - 넘파이 배열에 조건을 설정하면 True 또는 False를 요소로 하는 배열을 생성 → 조건 필터
  - True에 해당하는 요소만 추출
  - 대괄호에는 조건식뿐만 아니라 넘파이에서 제공하는 함수 사용 가능

```
data = np.array([4, 6, 7, 3, 2, 0])
print(data[data > 3])
```

```
print(data > 3)
cond = [False, True, True, False, False, True]
data[cond]
```

```
# 2보다 크고 6보다 작은 요소만 추출
print(data[(data > 3) & (data < 6)])
# 짝수만 출력
print(data[data % 2 == 0])
# 평균보다 작은 값만 추출
print(data[data < data.mean()])
```

※ 본 자료는 2026년 2학기 「SI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

17

17

## 팬시 인덱싱 (Fancy indexing)

### ■ 정수형 배열의 값을 사용하여 해당 정수의 인덱스에 위치한 값을 반환

배열명[list of index]

- 인덱스 항목에 넣을 배열은 정수로만 구성되어야 함
- 정수 값의 범위는 대상이 되는 배열이 가지는 인덱스의 범위 내
- 대상이 되는 배열과 인덱스 배열의 구조(shape)가 같을 필요는 없음

```
f = np.array([4, 6, 7, 3, 2])
indexes = np.array([1, 2, 0, 2, 2, 2])
f[indexes]
```

```
f2 = np.array([[1, 4], [9, 16]])
x = np.array([0, 1, 1, 1, 0, 0])
y = np.array([0, 0, 0, 1, 1, 1])
f2[x, y]
```

|   | 0 | 1  |
|---|---|----|
| 0 | 1 | 4  |
| 1 | 9 | 16 |

※ 본 자료는 2026년 2학기 「SI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

18

18

## 넘파이 배열의 정렬

### ■ 기본 정렬: sort 함수

- `sorted_ndarray = np.sort(ndarray)` : 정렬된 배열을 반환
- `ndarray.sort()` : in-place sort
- 오름차순 정렬만 지원
- 내림차순 정렬을 위한 slicing `::-1` 활용

```
numbers = np.array([2, 8, 9, 7, 3, 1, 4, 5, 6])
sorted_numbers = np.sort(numbers)
print(numbers)
print(sorted_numbers)
print(sorted_numbers[::-1])
```

```
numbers.sort()
print(numbers)
```

### ■ 2차원 배열의 정렬

- 매개변수 `axis`를 사용하여 행 또는 열 기준으로 정렬
- `axis = 0` : 행 방향으로 정렬
- `axis = 1` : 열 방향으로 정렬 (default)

axis=1 →

|          |   |    |    |    |
|----------|---|----|----|----|
|          | 1 | 2  | 3  | 4  |
| axis=0 ↓ | 5 | 6  | 7  | 8  |
|          | 9 | 10 | 11 | 12 |

```
arr2d=np.array([[5, 6, 7, 8],
                [4, 3, 2, 1],
                [10, 9, 12, 11]])
print(arr2d)
```

```
print(np.sort(arr2d))
print(np.sort(arr2d, axis=1))
print(np.sort(arr2d, axis=0))
```

※ 본 자료는 2026년 2학기 『시계열데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

19

19

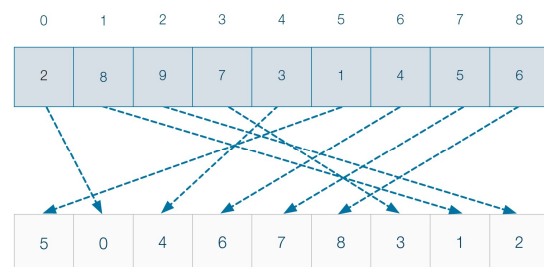
## 넘파이 배열의 정렬

### ■ 인덱스 반환 정렬 : argsort 함수

- `sorted_index_list = np.argsort(ndarray)`

- 정렬된 값의 index를 반환

```
numbers = np.array([2, 8, 9, 7, 3, 1, 4, 5, 6])
sorted_indexes = np.argsort(numbers)
print(sorted_indexes)
```



[연습문제] 이름과 성적이 각각의 ndarray로 주어진 경우 이름을 성적 순서로 출력하기

```
names = np.array(['John', 'Mike', 'Sarah', 'Kate', 'Samuel'])
scores= np.array([78, 95, 84, 98, 88])
```

※ 본 자료는 2026년 2학기 『시계열데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

20

20

## 넘파이 배열의 연산

- 넘파이는 파이썬과 동일하게 배열 간 사칙연산 지원
  - 행렬과 행렬, 벡터와 벡터 간 사칙연산이 가능
- 같은 배열의 구조일 때 요소별 연산 (element-wise operation)
  - 요소별 연산 : 두 배열의 구조가 동일할 경우 같은 인덱스 요소들끼리 연산

|   |    |    |    |
|---|----|----|----|
| 1 | 2  | 3  | 4  |
| 5 | 6  | 7  | 8  |
| 9 | 10 | 11 | 12 |

 $\times$ 

|   |    |    |    |
|---|----|----|----|
| 1 | 2  | 3  | 4  |
| 5 | 6  | 7  | 8  |
| 9 | 10 | 11 | 12 |

 $=$ 

|    |     |     |     |
|----|-----|-----|-----|
| 1  | 4   | 9   | 16  |
| 25 | 36  | 49  | 64  |
| 81 | 100 | 121 | 144 |

※ 본 자료는 2026년 2학기 『시계반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

21

21

## 넘파이 배열의 연산

### ■ 브로드캐스팅 (broadcasting)

- 모양(shape)이 다른 배열들 간에도 산술 연산이 가능하게 하는 기능
  - 방송국의 전파가 퍼지듯 뒤에 있는 모든 요소에 퍼지듯이 연산
  - 모양과 차원이 다른 배열 간의 조건(차원과 구조)을 만족하도록 배열을 변환하여 연산이 가능하도록 함
- 브로드캐스팅이 일어날 수 있는 조건
  - 최소한 하나의 배열의 차원이 1인 경우
  - 두 배열의 차원의 쌍이 맞을 경우
  - 차원에 대해 행의 길이 혹은 열의 길이가 같은 경우

|    |    |    |
|----|----|----|
| 1  | 2  | 3  |
| 4  | 5  | 6  |
| 7  | 8  | 9  |
| 10 | 11 | 12 |

 $+$ 

|   |
|---|
| 3 |
|---|

 $=$ 

|    |    |    |
|----|----|----|
| 4  | 5  | 6  |
| 7  | 8  | 9  |
| 10 | 11 | 12 |
| 13 | 14 | 15 |

※ 본 자료는 2026년 2학기 『시계반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

22

22

## 넘파이 배열의 연산

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + 3 = \begin{bmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

```
a = np.array([[1, 2, 3], [4, 5, 6]])
b = 3
print(a+b)
```

$$\begin{bmatrix} 0 & 0 & 0 \\ 10 & 10 & 10 \\ 20 & 20 & 20 \\ 30 & 30 & 30 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 10 & 10 & 10 \\ 20 & 20 & 20 \\ 30 & 30 & 30 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix}$$

```
a = np.array([[0, 0, 0], [10, 10, 10], [20, 20, 20], [30, 30, 30]])
b = np.array([0, 1, 2])
print(a+b)
```

$$\begin{bmatrix} 0 \\ 10 \\ 20 \\ 30 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 10 & 10 & 10 \\ 20 & 20 & 20 \\ 30 & 30 & 30 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix}$$

```
a = np.array([[0], [10], [20], [30]])
b = np.array([0, 1, 2])
print(a+b)
```

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

23

23

## 넘파이 배열의 주요 연산 함수

`ndarray.sum(axis = None)`  
`ndarray.mean(axis = None)`  
`ndarray.max(axis = None)`  
`ndarray.min(axis = None)`

- 넘파이 배열의 *sum, mean, max, min*을 축을 따라 구하는 함수

`ndarray.argmax(axis = None)`  
`ndarray.argmin(axis = None)`

- 넘파이 배열 데이터 중 *max, min*의 값이 있는 *index* 반환

`ndarray.all(axis = None)`  
`np.all(ndarray, axis = None)`

- 배열 내부의 모든 값이 참일 때는 *True*, 하나라도 참이 아닐 경우에는 *False*를 반환
- *and* 조건을 전체 요소에 적용

```
print(np.all(a > 3))
```

`ndarray.any(axis = None)`  
`np.any(ndarray, axis = None)`

```
print(np.any(a > 4))
```

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

24

24

## 넘파이 배열의 주요 연산 함수

```
# 같은 배열 내 요소의 합 구하기
a = np.array([[1, 2, 3], [4, 5, 6]])
print(np.sum(a, axis=0))
print(np.sum(a, axis=1))
print(a.sum())
```

### ▪ any(), all() 함수의 활용

```
# 1. 배열에 결측치가 하나라도 포함되어 있는지 확인
data = np.array([1, 2, np.nan, 4])
print(np.any(np.isnan(data))) # True

# 2. 두 배열이 완전히 동일한지 확인
arr1 = np.array([1, 2, 3])
arr2 = np.array([1, 2, 3])
print(np.all(arr1 == arr2)) # True (요소별 비교 후 전체 확인)
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석(이해정 교수)』 강의자료로 무단 배포 및 발행을 금합니다.

25